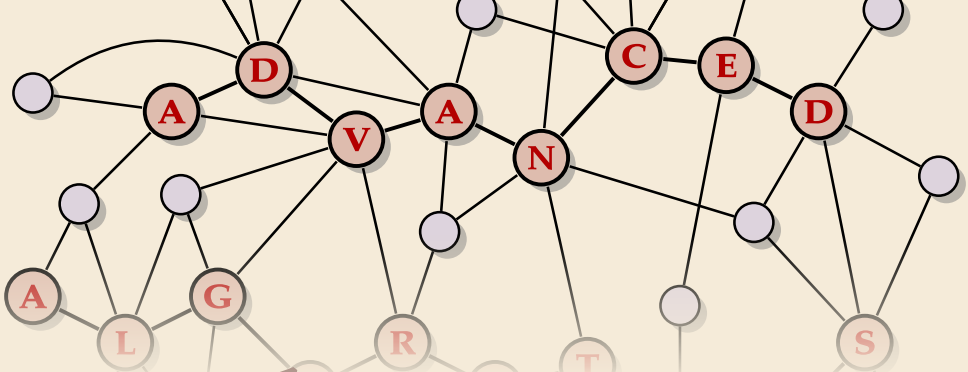


4 Fixed-Parameter Algorithms

Advanced Algorithms · Winter 2026–27

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
version `2026-11-02 14:47`

Outline

4 Fixed-Parameter Algorithms

4.1 Fixed-Parameter Tractability
4.2 Depth-Bounded Exhaustive Search I
4.3 Problem Kernels
4.4 Depth-Bounded Search II: Planar Independent Set
4.5 Depth-Bounded Search III: Closest String
4.6 Linear Recurrences & Better Vertex Cover
4.7 Interleaving

Philosophy of FPT

- **Goal:** Principled theory for studying complexity based on two dimensions: input size $n = |x|$ (encoding length) and *some additional parameter* k
  - generalize ideas from $k = MaxInt(x)$
  - investigate influence of k (and n) on running time

Philosophy of FPT

- **Goal:** Principled theory for studying complexity based on two dimensions: input size $n = |x|$ (encoding length) and *some additional parameter* k
  - generalize ideas from $k = MaxInt(x)$
  - investigate influence of k (and n) on running time

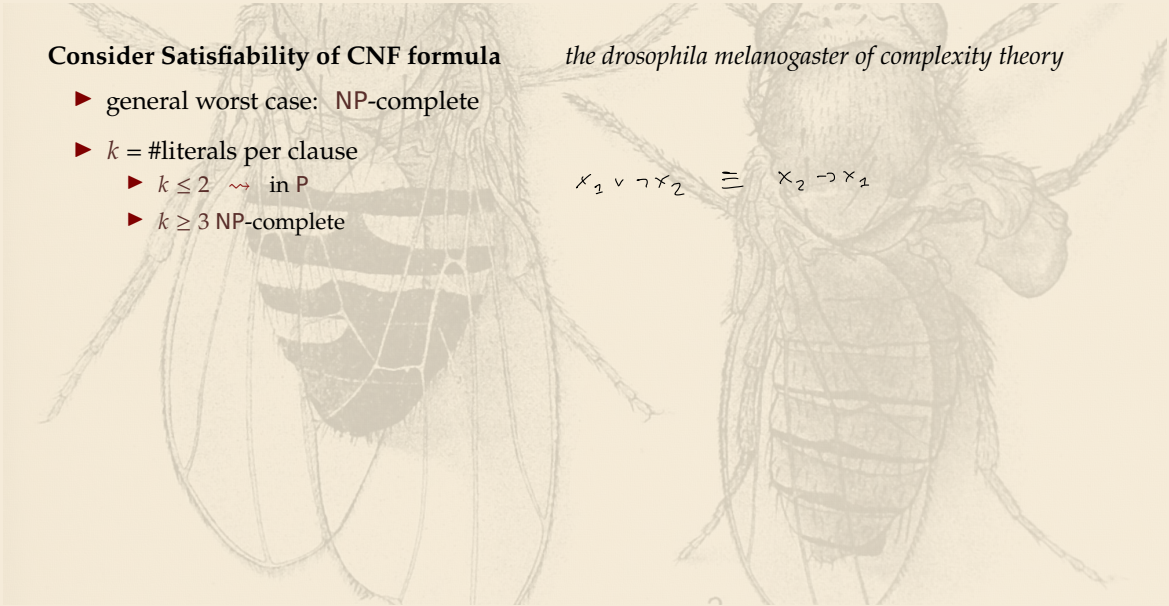
  $\rightsquigarrow$ Try to find a parameter k such that
  (1) the problem can be solved efficiently as long as k is small, and
  (2) practical instances have small values of k (even where n gets big).

Motivation: Satisfiability

Consider Satisfiability of CNF formula *the drosophila melanogaster of complexity theory*

- general worst case: NP-complete
- k = #literals per clause
  - $k \leq 2 \rightsquigarrow$ in P
  - $k \geq 3$ NP-complete

$x_1 \vee \neg x_2 \equiv x_2 \rightarrow x_1$



Motivation: Satisfiability

Consider Satisfiability of CNF formula *the drosophila melanogaster of complexity theory*

- general worst case: NP-complete
- k = #literals per clause
  - $k \leq 2 \rightsquigarrow$ in P
  - $k \geq 3$ NP-complete
- k = #variables
  - $O(2^k \cdot n)$ time possible (try all assignments)
- k = #clauses?
- k = #literals?
- k = #ones in satisfying assignment
- k = structural property of formula
- for MAX-SAT, k = #optimal clauses to satisfy

more reasonable

Parameters

Definition 4.1 (Parameterization)

Let $\Pi = (L, Y, \Sigma)$ be a decision problem. A *parameterization* of Π is a polytime computable mapping $\kappa : L \to \mathbb{N}$. ◀

Definition 4.2 (Parameterized problem)

A *parameterized (decision) problem* is a pair (Π, κ) of a decision problem Π and a parameterization κ of Π. ◀

Examples

As before: Typically leave encoding implicit.

Definition 4.3 (p-variables-SAT)

Given: formula boolean ϕ (same as before)

Parameter: number of variables

Question: Is there a satisfying assignment $v : [n] \to \{0,1\}$? ◀

Definition 4.4 (p-Clique)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Parameter: k

Question: $\exists V' \subseteq V \;:\; |V'| \geq k \;\wedge\; \forall u, v \in V' : \{u, v\} \in E$? ◀

Definition 4.5 (p-Vertex-Cover)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Parameter: k

Question: $\exists S \subseteq V \;:\; |S| \leq k \;\wedge\; \forall e \in E : S \cap e \neq \emptyset$? ◀

Canonical Parameterization

Typical use case: study (decision versions of) optimization problems

Definition 4.6 (Canonical Parameterization)

If the instances of a decision problem Π are pairs (x, k) with $k \in \mathbb{N}$, the *canonical parameterization* is $\kappa(x, k) = k$. We write $p\text{-}\Pi$ for (Π, κ).

For $\Pi = (\Sigma_I, \Sigma_O, L, L_I, M, \mathit{value}, \mathit{goal})$ an optimization problem, $p\text{-}\Pi$ denotes the *(canonically) parameterized (decision) problem* given by the threshold problem $\mathit{Threshold}_\Pi$:
$p\text{-}\Pi = p\text{-}\mathit{Threshold}_\Pi$. ◀

Recall: $\mathit{Threshold}_\Pi$ is the set of pairs (x, k) of all instances $x \in L_I$ that have solutions that are weakly "better" than k.

Canonical Parameterization

Typical use case: study (decision versions of) optimization problems

Definition 4.6 (Canonical Parameterization)

If the instances of a decision problem Π are pairs (x, k) with $k \in \mathbb{N}$, the *canonical parameterization* is $\kappa(x, k) = k$. We write $p\text{-}\Pi$ for (Π, κ).

For $\Pi = (\Sigma_I, \Sigma_O, L, L_I, M, \mathit{value}, \mathit{goal})$ an optimization problem, $p\text{-}\Pi$ denotes the *(canonically) parameterized (decision) problem* given by the threshold problem $\mathit{Threshold}_\Pi$:
$p\text{-}\Pi = p\text{-}\mathit{Threshold}_\Pi$. ◀

Recall: $\mathit{Threshold}_\Pi$ is the set of pairs (x, k) of all instances $x \in L_I$ that have solutions that are weakly "better" than k.

Examples:

- p-Clique
- p-Vertex-Cover
- p-Graph-Coloring
- ...

Naming convention for other parameters:
p-clause-CNF-SAT: CNF-SAT with parameter "number of *clauses*"

4.1 Fixed-Parameter Tractability

Exemplary Running Times of Parameterized Problems

- *p-variables*-SAT
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time

(consider simplest brute-force methods for problems)

Exemplary Running Times of Parameterized Problems

(consider simplest brute-force methods for problems)

- p-*variables*-SAT
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time
- p-Clique
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$

Exemplary Running Times of Parameterized Problems

- p-*variables*-SAT (consider simplest brute-force methods for problems)
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time
- p-Clique
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$
- p-VertexCover
  - k threshold (VC size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(m)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot m)$

Exemplary Running Times of Parameterized Problems

- *p-variables*-SAT (consider simplest brute-force methods for problems)
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time to get poly time $k = O(\log n)$ FPT

- p-Clique
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$ $k = O(1)$ for overall poly time

- p-VertexCover XP
  - k threshold (VC size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(m)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot m)$ $k = O(1)$

- p-GraphColoring
  - k threshold (#colors); n vertices, m edges in graph
  - $\rightsquigarrow$ k^n candidates to check, each takes time $O(m)$
  - $\rightsquigarrow$ Total time $O(k^n \cdot m)$ $k = 1$

$k = 3 \Rightarrow$ NP-hard

$k = 2 \in P$ (bipartite)

FPT Running Time

Definition 4.7 (fpt-algorithm)

Let (Π, κ) be a parameterized problem, $\Pi = (L, Y, \Sigma)$.
A (deterministic) algorithm A (with input alphabet Σ) is a *fixed-parameter tractable algorithm (fpt-algorithm)* w.r.t. κ if its running time on $x \in L$ with $\kappa(x) = k$ is at most

$$f(k) \cdot p(|x|) \quad = \quad O\big(f(k) \cdot |x|^c\big)$$

where p is a polynomial of degree c and f is an **arbitrary** computable function. ◀

FPT Running Time

Definition 4.7 (fpt-algorithm)

Let (Π, κ) be a parameterized problem, $\Pi = (L, Y, \Sigma)$.
A (deterministic) algorithm A (with input alphabet Σ) is a *fixed-parameter tractable algorithm (fpt-algorithm)* w.r.t. κ if its running time on $x \in L$ with $\kappa(x) = k$ is at most

$$f(k) \cdot p(|x|) \quad = \quad O\big(f(k) \cdot |x|^c\big)$$

where p is a polynomial of degree c and f is an **arbitrary** computable function. ◀

Definition 4.8 (FPT)

A parameterized problem (Π, κ) is *fixed-parameter tractable* if there is an fpt-algorithm w.r.t. κ that decides Π.
The complexity class of all such problems is denoted by FPT. ◀

Intuitively, FPT plays the role of P.

A First FPT Example

Theorem 4.9 (p-variables-SAT is FPT)

p-variables-SAT $\in$ FPT. ◀

A First FPT Example

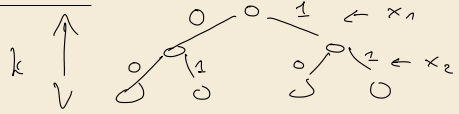
Theorem 4.9 (p-variables-SAT is FPT)

p-variables-SAT $\in$ FPT.

Proof:

Suffices to use brute force satisfiability for *p-variables*-SAT

```
1 procedure bruteForceSat(φ, X = {x_1, ..., x_k})
2     if k == 0
3         if φ == true return ∅ else UNSATISFIABLE
4     for value in {true, false} do
5         A := {x_1 ↦ value}
6         ψ := φ[x_1/value] // Substitute value for x_1
7         B := bruteForceSat(ψ, {x_2, ..., x_k})
8         if B ≠ UNSATISFIABLE
9             return A ∪ B
```



. . . but #variables not usually small

A First FPT Example

Theorem 4.9 (p-variables-SAT is FPT)

p-variables-SAT $\in$ FPT. ◀

Proof:
Suffices to use brute force satisfiability for *p-variables*-SAT

```
1 procedure bruteForceSat(φ, X = {x_1, ..., x_k})
2     if k == 0
3         if φ == true return ∅ else UNSATISFIABLE
4     for value in {true, false} do
5         A := {x_1 ↦ value}
6         ψ := φ[x_1/value] // Substitute value for x_1
7         B := bruteForceSat(ψ, {x_2, ..., x_k})
8         if B ≠ UNSATISFIABLE
9             return A ∪ B
```

Worst case running time: $O(2^k\, n)$ for $n = |\varphi|$.
2^k recursive calls;
base case needs time $O(|\phi|)$ to check whether formula evaluates to *true* ■

A First FPT Example

Theorem 4.9 (p-variables-SAT is FPT)

p-variables-SAT $\in$ FPT.

Proof:

Suffices to use brute force satisfiability for *p-variables*-SAT

```
1 procedure bruteForceSat(φ, X = {x_1, ..., x_k})
2     if k == 0
3         if φ == true return ∅ else UNSATISFIABLE
4     for value in {true, false} do
5         A := {x_1 ↦ value}
6         ψ := φ[x_1/value] // Substitute value for x_1
7         B := bruteForceSat(ψ, {x_2, ..., x_k})
8         if B ≠ UNSATISFIABLE
9             return A ∪ B
```

Worst case running time: $O(2^k\, n)$ for $n = |\varphi|$.

2^k recursive calls;

base case needs time $O(|\phi|)$ to check whether formula evaluates to *true*

. . . but #variables not usually small

Aren't we all FPT?

Theorem 4.10 (k never decreases → FPT)
Let $\Pi = (L, Y, \Sigma)$ be a decidable problem, $g : \mathbb{N} \to \mathbb{N}$ weakly increasing, unbounded and computable, and κ a parameterization of Π with

$$\forall x \in L : \kappa(x) \geq g(|x|).$$

Then $(\Pi, \kappa) \in \mathsf{FPT}$ (for *any* decidable Π!). ◀

g weakly increasing: $n \leq m \;\to\; g(n) \leq g(m)$
g unbounded: $\forall t\ \exists n \;:\; g(n) \geq t$

Proof:
Y decidable $\rightsquigarrow$ $\exists$ algorithm A deciding Π in time $\leq T(|x|)$ for *some* computable $T : \mathbb{N} \to \mathbb{N}$

w.l.o.g. T is weakly increasing and $T(n) \geq n$.

Aren't we all FPT?

Theorem 4.10 (k never decreases → FPT)
Let $\Pi = (L, Y, \Sigma)$ be a decidable problem, $g : \mathbb{N} \to \mathbb{N}$ weakly increasing, unbounded and computable, and κ a parameterization of Π with

$$\forall x \in L : \kappa(x) \geq g(|x|).$$

Then $(\Pi, \kappa) \in \mathsf{FPT}$ (for *any* decidable Π!). ◀

g weakly increasing: $n \leq m \;\to\; g(n) \leq g(m)$
g unbounded: $\forall t \; \exists n \; : \; g(n) \geq t$

Proof:
Y decidable $\rightsquigarrow$ $\exists$ algorithm A deciding Π in time $\leq T(|x|)$ for *some* computable $T : \mathbb{N} \to \mathbb{N}$

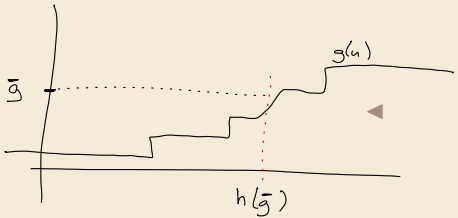
w.l.o.g. T is weakly increasing and $T(n) \geq n$.

Idea: "hide" $T(|x|)$ inside $f(k)$: since $k = \kappa(x) \geq g(|x|)$ and g is unbounded, a large input forces a large parameter.

Aren't we all FPT?

Theorem 4.10 (k never decreases → FPT)
Let $\Pi = (L, Y, \Sigma)$ be a decidable problem, $g : \mathbb{N} \to \mathbb{N}$ weakly increasing, unbounded and computable, and κ a parameterization of Π with

$$\forall x \in L : \kappa(x) \geq g(|x|).$$



Then $(\Pi, \kappa) \in \mathsf{FPT}$ (for *any* decidable Π!).

g weakly increasing: $n \leq m \;\to\; g(n) \leq g(m)$
g unbounded: $\forall t\ \exists n \;:\; g(n) \geq t$

Proof:
Y decidable $\rightsquigarrow$ $\exists$ algorithm A deciding Π in time $\leq T(|x|)$ for *some* computable $T : \mathbb{N} \to \mathbb{N}$
w.l.o.g. T is weakly increasing and $T(n) \geq n$.

Idea: "hide" $T(|x|)$ inside $f(k)$: since $k = \kappa(x) \geq g(|x|)$ and g is unbounded, a large input forces a large parameter.

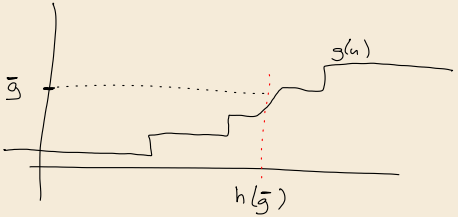
$\rightsquigarrow$ Define $h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \leq k \,\} \cup \{1\}\big)$.

Aren't we all FPT? – Proof

Proof (cont.):

$h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \leq k \,\} \cup \{1\}\big).$

1. g weakly increasing and unbounded $\rightsquigarrow$ the set is finite $\rightsquigarrow$ h well-defined ✓
2. h weakly increasing
3. g computable $\rightsquigarrow$ h computable (test $n = 1, 2, \ldots$ until $g(n) > k$)
4. $h(g(n)) \geq n$ for all n (as n is in the set for $k = g(n)$)



Aren't we all FPT? – Proof

Proof (cont.):

$h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \le k \,\} \cup \{1\}\big).$

1. g weakly increasing and unbounded $\rightsquigarrow$ the set is finite $\rightsquigarrow$ h well-defined
2. h weakly increasing
3. g computable $\rightsquigarrow$ h computable (test $n = 1, 2, \ldots$ until $g(n) > k$)
4. $h(g(n)) \ge n$ for all n (as n is in the set for $k = g(n)$)

Now let x be an input, $n = |x|$, $k = \kappa(x) \ge g(n)$. Running A on x takes time

$$T(n) \;\le\; T\big(h(g(n))\big) \;\le\; T\big(h(k)\big) \;=:\; f(k),$$

using (4) and T increasing, then $g(n) \le k$ and h, T increasing.

Aren't we all FPT? – Proof

Proof (cont.):

$h(k) \;:=\; \max\big(\{\, n \in \mathbb{N} : g(n) \le k \,\} \cup \{1\}\big).$

1. g weakly increasing and unbounded $\rightsquigarrow$ the set is finite $\rightsquigarrow$ h well-defined
2. h weakly increasing
3. g computable $\rightsquigarrow$ h computable (test $n = 1, 2, \ldots$ until $g(n) > k$)
4. $h(g(n)) \ge n$ for all n (as n is in the set for $k = g(n)$)

Now let x be an input, $n = |x|$, $k = \kappa(x) \ge g(n)$. Running A on x takes time

$$T(n) \;\le\; T\big(h(g(n))\big) \;\le\; T\big(h(k)\big) \;=:\; f(k),$$

using (4) and T increasing, then $g(n) \le k$ and h, T increasing.

f is computable, so A is an fpt-algorithm for (Π, κ) (with $p(n) \equiv 1$). ■

Back to “sensible” parameters

- ⇝ always check if parameter is reasonable (can be expected to be small)
  - if not, FPT might not even mean in NP!

Back to “sensible” parameters

- ⇝ always check if parameter is reasonable (can be expected to be small)
  - if not, FPT might not even mean in NP!
- but now, for some positive examples!

4.2 Depth-Bounded Exhaustive Search I

FPT Design Pattern

- The simplest FPT algorithms use exhaustive search
- but with a search tree bounded by $f(k)$

FPT Design Pattern

- The simplest FPT algorithms use exhaustive search
- but with a search tree bounded by $f(k)$

- bruteforceSat was a typical example!
- does this work on other problems?

Depth-Bounded Search for Vertex Cover

Let's try p-VertexCover.

Key insight: for every edge $\{v, w\}$, any vertex cover must contain v or w

Depth-Bounded Search for Vertex Cover

Let's try p-VertexCover.

Key insight: for every edge $\{v, w\}$, any vertex cover must contain v or w

```
1  procedure simpleFptVertexCover(G = (V, E), k):
2      if E == ∅ then return ∅
3      if k == 0 then return NOT_POSSIBLE  // truncate search
4      Choose {v, w} ∈ E (arbitrarily)
5      for u in {v, w} do:
6          G_u := (V \ {u}, E \ {{u, x} ∈ E})  // Remove u from G
7          C_u := simpleFptVertexCover(G_u, k − 1)
8      if C_v == NOT_POSSIBLE then return C_w ∪ {w}
9      if C_w == NOT_POSSIBLE then return C_v ∪ {v}
10     if |C_v| ≤ |C_w| then return C_v ∪ {v} else return C_w ∪ {w}
```

Depth-Bounded Search for Vertex Cover

Let's try p-VertexCover.
Key insight: for every edge $\{v, w\}$, any vertex cover must contain v or w

```
1  procedure simpleFptVertexCover(G = (V, E), k):
2      if E == ∅ then return ∅
3      if k == 0 then return NOT_POSSIBLE  // truncate search
4      Choose {v, w} ∈ E (arbitrarily)
5      for u in {v, w} do:
6          G_u := (V \ {u}, E \ {{u, x} ∈ E})  // Remove u from G
7          C_u := simpleFptVertexCover(G_u, k − 1)
8      if C_v == NOT_POSSIBLE then return C_w ∪ {w}
9      if C_w == NOT_POSSIBLE then return C_v ∪ {v}
10     if |C_v| ≤ |C_w| then return C_v ∪ {v} else return C_w ∪ {w}
```

induced subgraph of G (by subset of vertices $V \setminus \{u\}$)

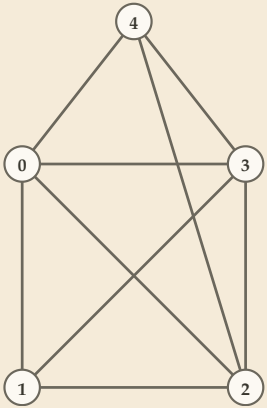
$G_u = G[V \setminus \{u\}]$

- Does not need explicit checks of solution candidates!
- runs in time $O(2^k(n+m))$ $\rightsquigarrow$ fpt-algorithm for p-Vertex-Cover

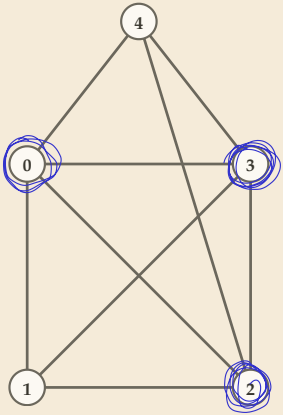
$T(n,k) \leq 2\,T(n,k-1) + \Theta(n) \quad \Rightarrow \quad T(n,k) = 2^k \cdot \Theta(n)$

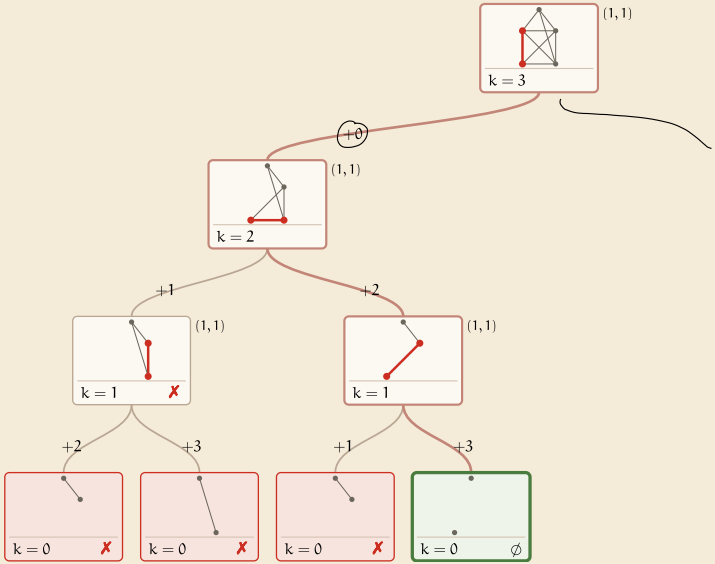
Note: $\binom{n}{k} = \Theta(n^k) \gg 2^k$

Depth-Bounded Search for Vertex Cover – Example



Depth-Bounded Search for Vertex Cover – Example





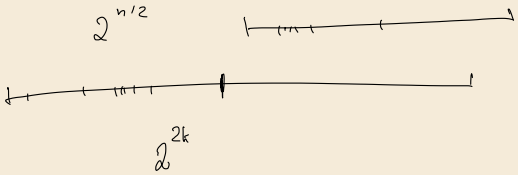
Guessing the parameter

- Note: Previous algorithm only uses k to *truncate* branches.

⇝ We can *guess* a k and it still works

binary search too expensive !

exponential search



linear search 2^k

Guessing the parameter

- Note: Previous algorithm only uses k to *truncate* branches.

$\rightsquigarrow$ We can *guess* a k and it still works

$\rightsquigarrow$ Try all k!

```
1 procedure vertexCoverBfs(G = (V, E))
2     for k := 0, 1, ..., |V| do
3         C := simpleFptVertexCover(G, k)
4         if C ≠ NOT_POSSIBLE return C
```

Guessing the parameter

- Note: Previous algorithm only uses k to *truncate* branches.

⇝ We can *guess* a k and it still works

⇝ Try all k!

```
1 procedure vertexCoverBfs(G = (V, E))
2     for k := 0, 1, ..., |V| do
3         C := simpleFptVertexCover(G, k)
4         if C ≠ NOT_POSSIBLE return C
```

- Running time: $\sum_{k'=0}^{k} O(2^{k'}(n+m)) \;=\; O(2^{k}(n+m))$

⇝ For exponentially growing cost, trying all values up to k costs only constant factor more

4.3 Problem Kernels

Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome



Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set x to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case ...

Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set x to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case ...
  - special case of resolution calculus rule $\dfrac{a_1 \vee a_2 \vee \cdots \vee x, \quad b_1 \vee b_2 \vee \cdots \vee \neg x}{a_1 \vee a_2 \vee \cdots \vee b_1 \vee b_2 \vee \cdots}$
  - basis of practical SAT solvers

Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set x to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case ...
  - special case of resolution calculus rule $\dfrac{a_1 \vee a_2 \vee \cdots \vee x, \quad b_1 \vee b_2 \vee \cdots \vee \neg x}{a_1 \vee a_2 \vee \cdots \vee b_1 \vee b_2 \vee \cdots}$
  - basis of practical SAT solvers
- Trivial example for VertexCover

  Remove vertices of degree 0 or 1.

  include other endpoint in solution

  (never needed as part of optimal VC)

Preprocessing

- Second key fpt technique are *reduction rules*
- **Idea:** Reduce the size of the instance (in polytime) without changing its outcome
- Trivial example for SAT:

  If a CNF formula contains a single-literal clause $\{x\}$ resp. $\{\neg x\}$, set x to *true* resp. *false* and remove the clause.

  - doesn't do anything in the worst case ...
  - special case of resolution calculus rule $\dfrac{a_1 \vee a_2 \vee \cdots \vee x, \quad b_1 \vee b_2 \vee \cdots \vee \neg x}{a_1 \vee a_2 \vee \cdots \vee b_1 \vee b_2 \vee \cdots}$
  - basis of practical SAT solvers
- Trivial example for VertexCover

  Remove vertices of degree 0 or 1.

  include other endpoint in solution

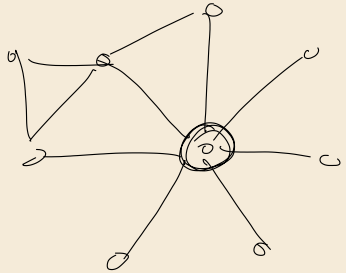
  (never needed as part of optimal VC)
- Here: reduction rules that provably shrink an instance to size $g(k)$

Buss's Reduction Rule for VC

- Given a p-VertexCover instance (G, k)

  "deg > k" Rule: If G contains vertex v of degree $\deg(v) > k$, include v in potential solution and remove it from the graph.

- Can apply this simultaneously to degree $> k$ vertices.
- Either rule applies, or all vertices bounded degree(!)



Kernels

preprocessing
=

Definition 4.11 (Kernelization)

Let (Π, κ) be a parameterized problem, $\Pi = (L, Y, \Sigma)$.
A function $K : L \to L$ is a *kernelization* of (Π, κ) if it maps every instance $x \in L$ with $k = \kappa(x)$ to an instance $x' = K(x)$ so that

1. (self-reduction) $x \in Y \iff x' \in Y$
2. (polytime) K is computable in polytime.
3. (kernel-size) $|x'| \leq g(k)$ and $\kappa(x') \leq g(k)$ for some computable function g

We call x' the *(problem) kernel* of x and g the *size of the problem kernel*. ◀

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)
Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)

Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Proof:

After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.

(Note that the rule might reduce the parameter k).

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)
Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Proof:
After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.
(Note that the rule might reduce the parameter k).
In the resulting graph, any VC of size $\leq k$ covers $\leq k^2$ edges.

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)

Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Proof:

After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.
(Note that the rule might reduce the parameter k).
In the resulting graph, any VC of size $\leq k$ covers $\leq k^2$ edges.
If $m > k^2$, we output a trivial No-instance (e. g., a K_{k+1} a complete graph on $k+1$ vertices).

Buss's Kernel

Buss's Reduction for Vertex Cover: (repeatedly apply until no more changes)

- $\deg > k$ rule
- Remove degree 0 and 1 vertices

Theorem 4.12 (Buss's Reduction is Kernelization)
Buss' reduction yields a kernelization for p-Vertex-Cover with kernel size $O(k^2)$. ◀

Proof:
After repeatedly applying Buss's rule as well as the isolated/leaf rule until neither applies further, we have $\forall v \in V : 2 \leq \deg(v) \leq k$.



(Note that the rule might reduce the parameter k).
In the resulting graph, any VC of size $\leq k$ covers $\leq k^2$ edges.
If $m > k^2$, we output a trivial No-instance (e. g., a K_{k+1} a complete graph on $k+1$ vertices).
If $m \leq k^2$, then the input size is now bounded by $g(k) = 2k^2$. ■

FPT iff Kernelization

Theorem 4.13 (FPT ↔ kernel)

Let (Π, κ) be a parameterized problem with Π decidable.
(Π, κ) is fixed-parameter tractable if and only if there is a kernelization of (Π, κ).

Proof:

"$\Leftarrow$" kernelization K for (Π, κ) given. $\forall: x \in L \quad x \in Y \Leftrightarrow K(x) \in Y$

$|K(x)| \leq g(\underbrace{\kappa(x)}_{k})$

Y has algorithm A deciding $x \in Y$ in running time $T(n)$, wlog T weakly increasing

Construct fpt-algorithm for (Π, κ)

(1) given $x \in L$, compute $x' := K(x)$ in poly time

$|x'| \leq g(k)$

(2) run A on x'

time $T(|x'|) \leq T(g(k)) =: f(k)$

$\Rightarrow$ total time for deciding $x \in Y$ is $\mathcal{O}(f(k) + n^c)$ for constant c

FPT iff Kernelization [2]

Proof (cont.):

"$\Rightarrow$" Given fpt-algorithm A for (Π, κ) with time $\leq f(k) \cdot n^c$

(1) Simulate A for $\leq n^{c+1}$ steps (poly time)

(2) If A terminates before n^{c+1} steps

if answer from A was "yes": output trivial yes-instance

if answer from A was "no": output trivial no-instance

Otherwise $n^{c+1} \leq f(k) \cdot n^c \Rightarrow n \leq f(k)$

output original input.

Max-SAT Kernel

Theorem 4.14 (Kernel for Max-SAT)

p-Max-SAT has a problem kernel of size $O(k^2)$ which can be constructed in linear time.

Proof: Given formula φ with m clauses, can we satisfy $\geq k$ clauses

wlog. every variable occurs at most once per clause

($x \vee \neg x \in C \rightsquigarrow$ drop C, always satisfied)

n = total # literals in all clauses (try to make $n = O(k^2)$)

Case 1 $k \leq \lfloor \frac{m}{2} \rfloor$ Output true

Pick arbitrary assignment of all variables

$\Rightarrow \ell$ clauses satisfied if $\ell \geq k$ ✓

otherwise $\ell < k \leq \lfloor \frac{m}{2} \rfloor$ ($\Leftrightarrow \ell \leq \frac{m}{2}$)

negating all variables $\Rightarrow m - \ell \geq m - \lfloor \frac{m}{2} \rfloor \geq \lfloor \frac{m}{2} \rfloor \geq k$

clauses satisfied

Max-SAT Kernel [2]

Proof (cont.):

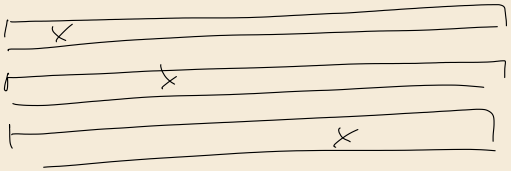
Case 2: $k > \lfloor \frac{m}{2} \rfloor \Rightarrow k > \frac{m}{2} \Rightarrow \boxed{m < 2k}$

(few clauses but n could still be large)

$F_L := \{$ clauses C : C has $\geq k$ literals $\}$ $\quad L := |F_L|$

$F_S := \{$ —— —————— $< k$. $\}$

2a: $\boxed{L \geq k}$



$\exists$ disjoint set of k variables that satisfies k long clauses

$\Rightarrow$ return true

Max-SAT Kernel [3]

< 2k clauses
< k literals in F_S
$\Rightarrow O(k^2)$ kernel

Proof (cont.):

bounded in size by k

25 $L < k$

Claim: $(F_S, k-L)$ is Yes-instance

$\Leftrightarrow (F, k)$ is ——

Proof: "$\Leftarrow$" trivial

"$\Rightarrow$" If A satisfies $k-L$ clauses in F_S

each of the $k-L$ clauses contains a true literal

$\Rightarrow$ $k-L$ variables make all of F_S true

$\Rightarrow$ can find L unset variables to make four clauses true.

Corollary 4.15

p-Max-SAT $\in$ FPT

4.4 Depth-Bounded Search II: Planar Indep. Set

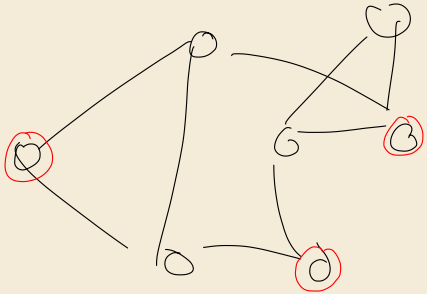
Deeper results (towards more shallow trees)

- Our previous examples of depth-bounded search were basically brute force
- Here we will see two more examples that exploit the problem structure in more interesting ways

Deeper results (towards more shallow trees)

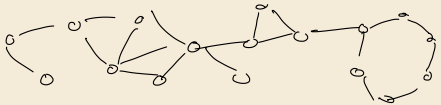
- Our previous examples of depth-bounded search were basically brute force
- Here we will see two more examples that exploit the problem structure in more interesting ways

- Recall: IndependentSet



Deeper results (towards more shallow trees)

- Our previous examples of depth-bounded search were basically brute force
- Here we will see two more examples that exploit the problem structure in more interesting ways



- Recall: IndependentSet
- IndependentSet is NP-hard on planar graphs even with vertex degrees at most 3
- We will see: general problem p-Independent-Set likely not in FPT.

Independent Set on Planar Graphs

Definition 4.16 (p-Planar-Independent-Set)

Given: a *planar* graph $G = (V, E)$ and $k \in \mathbb{N}$
Parameter: k
Question: $\exists V' \subset V \quad : \quad |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$?

◀

Independent Set on Planar Graphs

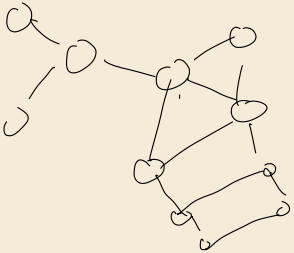
Definition 4.16 (p-Planar-Independent-Set)

Given: a *planar* graph $G = (V, E)$ and $k \in \mathbb{N}$
Parameter: k
Question: $\exists V' \subset V \;:\; |V'| \geq k \;\wedge\; \forall u, v \in V' : \{u, v\} \notin E$?

Theorem 4.17 (Depth-Bounded Search for Planar Independent Set)

p-Planar-Independent-Set is in FPT and can be solved in time $O(6^k n)$.

Planar graph G : $\exists$ embedding in plane ($x, y : V \to \mathbb{R}$, v at $(x(v), y(v))$)
way to draw all edges w/o crossings

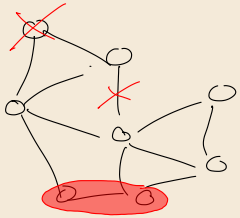


Wagner's Theorem: G planar iff does not contain K_5 or $K_{3,3}$ as minor

K_5  $K_{3,3}$ 

Minor: start with G and do

- vertex deletions
- edge deletions
- edge contractions



Every minor of a planar graph is planar.

Elementary Knowledge on Planar Graphs

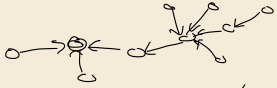
Theorem 4.18 (Euler's formula)

In any finite, *connected* (embedded) planar graph G with n vertices, m edges, and f faces holds $n - m + f = 2$.

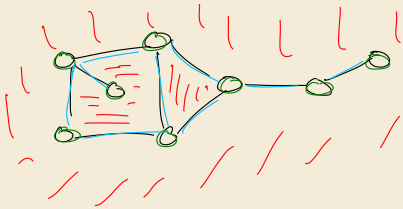
Proof idea: Induction on f

IB: $f = 1$ $\Rightarrow$ no cycles!

$\Rightarrow$ G is a tree



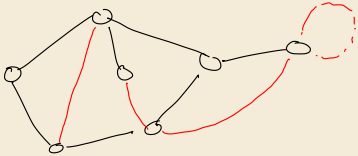
$\Rightarrow$ $m = n - 1$ ✓



IS: "$f \to f+1$" w/o new vertices

add one face

$\Rightarrow$ add one edge



Elementary Knowledge on Planar Graphs

Theorem 4.18 (Euler's formula)

In any finite, *connected* (embedded) planar graph G with n vertices, m edges, and f faces holds $n - m + f = 2$. ◀

$f = 2 + m - n$

Corollary 4.19

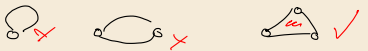
(i) A *simple* planar graph G on $n \geq 3$ nodes has $m \leq 3n - 6$ edges.

(ii) The average degree in G is < 6. ◀

(ii) $\sum_v \deg(v) = 2m \leq 6n - 12 \quad | : n$

$\text{avg deg} \leq 6 - \frac{12}{n} < 6$

(i) smallest face ?



Summing adjacent edges over all faces $\geq 3 \cdot f$

" counts every edge twice

$3f \leq 2m$

$\Rightarrow \quad m \leq 3n - 6$

$3(2 + m - n) = 6 + 3m - 3n$

Can say more here: avg deg $< 6 \Rightarrow \exists v$, $\deg(v) \leq 5$

↗ true for *every* simple planar graph

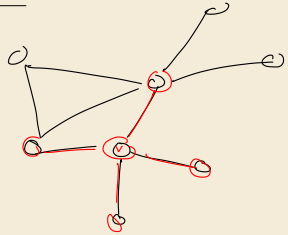
so can find a new ≤ 5 degree vertex after deleting the first!

$\Rightarrow$ Planar graphs are *5-degenerate*

" every induced subgraph contain vertex deg ≤ 5

Depth-Bounded Search for Planar Independent Set

```
1  procedure planarIndependentSet(G = (V, E), k):
2      if k == 0 then return ∅
3      if k > |V| then return NOT_POSSIBLE // truncate search
4      Choose v ∈ V with minimal degree; let w_1, ..., w_d be v's neighbors
5      // By planarity, we know d ≤ 5.
6      for u in {v, w_1, ..., w_d} do    // wlog any maximal IS contains one of them
7          D := {u} ∪ N(u)
8          G_u := (V \ D, E \ {{x, y} ∈ E : x ∈ D}) // Delete u and its neighbors
9          I'_u := planarIndependentSet(G_u, k − 1)
10         if I'_u ≠ NOT_POSSIBLE then return {u} ∪ I'_u
11     return NOT_POSSIBLE
```



≤ 6 recursive calls (for G 5-degenerate)

⇒ 6^k recursive calls in worst case

(each takes $\Theta(n+m) = \Theta(n)$ time)

⇒ total $O(6^k \cdot n)$

Summary Planar Independent Set

- planarIndependentSet will often be faster than $O(6^k n)$
- works unchanged in $O((d+1)^k n)$ time for any degeneracy-d graph

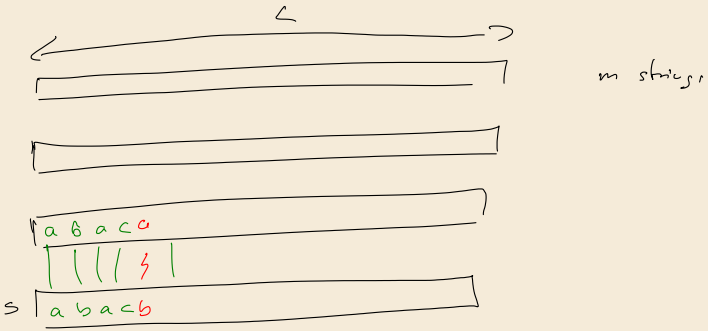
every (induced) subgraph has vertex of degree at most d

4.5 Depth-Bounded Search III: Closest String

Closest String

Definition 4.20 (p-Closest-String)

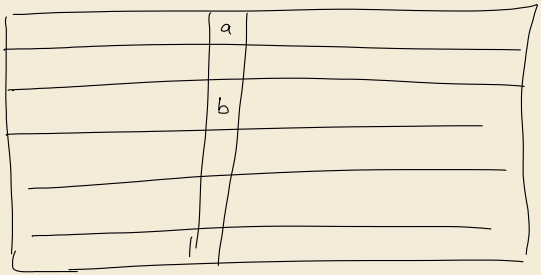
Given: S set of m strings $s_1, s_2, \ldots, s_m$ of length L over alphabet Σ and a $k \in \mathbb{N}$.
Parameter: k
Question: Is there a string s for which $d_H(s, s_i) \leq k$ holds for all $i = 1, \ldots, m$?



Dirty Columns

Definition 4.21 (Dirty Column)

A column of the $m \times L$ matrix corresponding to m strings of length L is called *dirty* if it contains at least 2 different symbols.



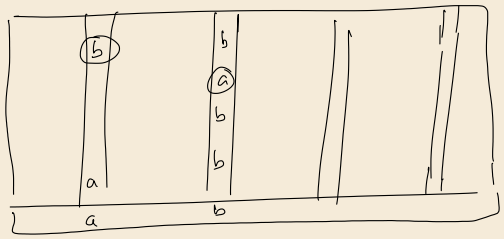
Dirty Columns

Definition 4.21 (Dirty Column)

A column of the $m \times L$ matrix corresponding to m strings of length L is called *dirty* if it contains at least 2 different symbols. ◀

Lemma 4.22 (Many Dirty Columns → No)

Let an instance to Closest-String with m strings of length L and parameter k be given.
If the corresponding $m \times L$ matrix contains more than $m \cdot k$ dirty columns, then no solution for the given instance exists. ◀



Depth-Bounded Search for Closest String

```
1  procedure closestStringFpt(s, d):
2      if d < 0 then return NOT_POSSIBLE
3      if d_H(s, s_i) > k + d for an i ∈ {1, ..., m} then
4          return NOT_POSSIBLE
5      if d_H(s, s_i) ≤ k for all i = 1, ..., m then return s
6      Choose i ∈ {1, ..., m} arbitrarily with d_H(s, s_i) > k
7          P := {p : s[p] ≠ s_i[p]}
8          Choose arbitrary P' ⊆ P with |P'| = k + 1
9          for p in P' do
10             s' := s
11             s'[p] := s_i[p]
12             s_ret := closestStringFpt(s', d − 1)
13             if s_ret ≠ NOT_POSSIBLE then return s_ret
14     return NOT_POSSIBLE
```

d = walking away from s_1 budget

search space $(k+1)^k$

$= O(k^k)$

$$\lim_{k\to\infty} \frac{(k+1)^k}{k^k} = \lim_{k\to\infty} \left(1 + \frac{1}{k}\right)^k$$

$= e < \infty$

- initial call closestStringFpt(s_1, k)

$$\lim_{n\to\infty} \left(1 + \frac{x}{n}\right)^n = e^x.$$

$$\left(1 + \tfrac{1}{n}\right)^n < e < \left(1 + \tfrac{1}{n}\right)^{n+1}.$$

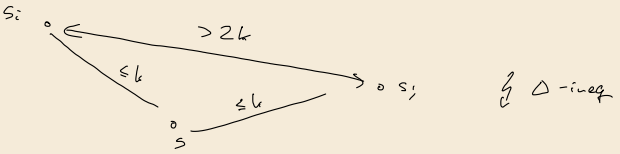
Too Much Dirt

Lemma 4.23 (Pair Too Different → No)
Let $S = \{s_1, s_2, \ldots, s_m\}$ a set of strings and $k \in \mathbb{N}$. If there are $i, j \in \{1, \ldots, m\}$ with $d_H(s_i, s_j) > 2k$, then there is no string s with $\max_{1 \le i \le m} d_H(s, s_i) \le k$. ◀

Proof: d_H is a metric, in particular it satisfies Δ-ineq.

x, y, z $\quad d_H(x,z) \le d_H(x,y) + d_H(y,z)$ ⇑

d_H = count edit operations



Depth-Bounded Search for Closest String

Theorem 4.24 (Search Tree for Closest String)

There is a search tree of size $O(k^k)$ for problem p-Closest-String. ◀

Depth-Bounded Search for Closest String

Theorem 4.24 (Search Tree for Closest String)

There is a search tree of size $O(k^k)$ for problem p-Closest-String. ◀

Corollary 4.25 (Closest String is FPT)

p-Closest-String can be solved in time $O(mL + mk \cdot k^k)$. ◀

- preprocessing ($O(mL)$ time)
  - ignore any clean columns
  - reject if more than mk dirty columns

⇝ effective string length after preprocessing is $L' \leq mk$

- call closestStringFpt(s_1, k)
  - maintain $d_H(s, s_i)$ in an array
    - ⇝ checking any distance $d_H(s, s_i)$ takes $O(1)$ time
    - before and after recursive call, update array to reflect $d_H(s', s_i)$
      Single character changed, so update only needs to check single position
    - ⇝ Can maintain distances in $O(m)$ time per recursive call
  - P' can be computed in $O(mk)$ time